

 c0c250019f / ProjExD_2



[Code](#) [Issues](#) [1](#) [Pull requests](#) [Agents](#) [Actions](#) [Projects](#) [Wiki](#) [Security and quality](#) [Insights](#)

[Settings](#)

  main [ProjExD_2 / dodge_bomb.py](#) 

 Go to file  

 c0c250019f 改行修正 #1

e5443f8 · 1 分前 

[Code](#) [Blame](#) 172 lines (150 loc) · 5.54 KB

[Raw](#) 

```
1  import time
2  import random
3  import os
4  import sys
5  import pygame as pg
6
7
8  WIDTH, HEIGHT = 1100, 650
9  os.chdir(os.path.dirname(os.path.abspath(__file__)))
10
11
12  ▼ DELTA = {
13      pg.K_UP: (0, -5),
14      pg.K_DOWN: (0, 5),
15      pg.K_LEFT: (-5, 0),
16      pg.K_RIGHT: (5, 0),
17  }
18
19
20  ▼ def check_bound(rect:pg.Rect) -> tuple[bool, bool]:
21      """
22      引数：こうかとんRectかばくだんRect
23      戻り値：タプル（横方向判定結果, 縦方向判定結果）
24      画面内ならTrue,画面外ならFalse
25      """
26      yoko, tate = True, True
27      if rect.left < 0 or rect.right > WIDTH:
28          yoko = False
29      if rect.top < 0 or rect.bottom > HEIGHT:
30          tate = False
31      return yoko, tate
32
33
34  ▼ def gameover(screen: pg.Surface) -> None:
35      """
36      引数：からのSurface
37      戻り値：なし
38      画面をブラックアウトしてGame Overの文字列を5秒表示
39      """
40      go_img = pg.Surface((WIDTH, HEIGHT))
41      go_img.set_alpha(200)
```

```

42     fonto = pg.font.Font(None, 80)
43     text = fonto.render("Game Over", True, (255, 255, 255))
44     k8_ing = pg.image.load("fig/8.png")
45     screen_center = WIDTH / 2, HEIGHT / 2
46     go_img.blit(text, (text.get_rect(center=screen_center)))
47     go_img.blit(k8_ing, (310, 290))
48     go_img.blit(k8_ing, (760, 290))
49     screen.blit(go_img, (0, 0))
50     pg.display.update()
51     time.sleep(5)
52
53
54     def init_bb_imgs() -> tuple[list[pg.Surface], list[int]]:
55         """
56         引数：なし
57         戻り値：爆弾の大きさ、爆弾の速度のリストをまとめたタプル
58         爆弾のサイズと速度を変更して返す
59         """
60         bb_imgs = []
61         bb_rads = []
62         for r in range(1, 11):
63             bb_img = pg.Surface((20*r, 20*r))
64             pg.draw.circle(bb_img, (255, 0, 0), (10*r, 10*r), 10*r)
65             bb_img.set_colorkey((0,0,0))
66             bb_imgs.append(bb_img)
67             bb_accs = [a for a in range(1, 11)]
68         return bb_imgs, bb_accs
69
70
71     def get_kk_imgs() -> dict[tuple[int, int], pg.Surface]:
72         """
73         引数：なし
74         戻り値：移動量タプルと対応する画像Surfaceの辞書
75         移動量に応じて向きと角度を変更した画像を返す
76         """
77
78         kk_dict = {
79             (0, 0): pg.transform.rotozoom(pg.image.load("fig/3.png"), 0, 0.9),
80             (+5, 0): pg.transform.rotozoom(pg.transform.flip(pg.image.load("fig/3.png"), True, False), 0, 0.9),
81             (+5, -5): pg.transform.rotozoom(pg.transform.flip(pg.image.load("fig/3.png"), True, False), 45, 0.9),
82             (0, -5): pg.transform.rotozoom(pg.transform.flip(pg.image.load("fig/3.png"), True, False), 90, 0.9),
83             (-5, -5): pg.transform.rotozoom(pg.image.load("fig/3.png"), -45, 0.9),
84             (-5, 0): pg.transform.rotozoom(pg.image.load("fig/3.png"), 0, 0.9),
85             (-5, +5): pg.transform.rotozoom(pg.image.load("fig/3.png"), 45, 0.9),
86             (0, +5): pg.transform.rotozoom(pg.transform.flip(pg.image.load("fig/3.png"), True, False), -90, 0.9),
87             (+5, +5): pg.transform.rotozoom(pg.transform.flip(pg.image.load("fig/3.png"), True, False), -45, 0.9),
88         }
89         return kk_dict
90
91
92     def calc_orientation(org: pg.Rect, dst: pg.Rect,
93                        current_xy: tuple[float, float]) -> tuple[float, float]:
94         """
95         引数：爆弾のRectとこうかとんのRectと現在の移動方向
96         戻り値：爆弾の移動方向
97         orgからdstへの差ベクトルを求め、ベクトルの長さがv50になるように正規化する。距離が300未満なら現在の移動方向をその
98         """
99         dx = dst.centerx - org.centerx

```

```
100     dy = dst.centery - org.centery
101     norm = (dx**2 + dy**2) ** 0.5
102     if norm < 300:
103         return current_xy
104     target_norm = 50 ** 0.5
105     vx = dx / norm * target_norm
106     vy = dy / norm * target_norm
107     return vx, vy
108
109
110  ✓ def main():
111     pg.display.set_caption("逃げろ！こうかとん")
112     screen = pg.display.set_mode((WIDTH, HEIGHT))
113     bg_img = pg.image.load("fig/pg_bg.jpg")
114     kk_img = pg.transform.rotozoom(pg.image.load("fig/3.png"), 0, 0.9)
115     kk_rct = kk_img.get_rect()
116     kk_rct.center = 300, 200
117     bb_img = pg.Surface((20, 20))
118     pg.draw.circle(bb_img, (255, 0, 0), (10, 10), 10)
119     bb_rct = bb_img.get_rect()
120     bb_rct.center = random.randint(0, WIDTH), random.randint(0, HEIGHT)
121     bb_img.set_colorkey((0, 0, 0))
122     bb_imgs, bb_accs = init_bb_imgs()
123     vx, vy = +5, +5
124     tmr = 0
125     clock = pg.time.Clock()
126     kk_imgs = get_kk_imgs()
127     while True:
128         for event in pg.event.get():
129             if event.type == pg.QUIT:
130                 return
131         screen.blit(bg_img, [0, 0])
132
133         key_lst = pg.key.get_pressed()
134         sum_mv = [0, 0]
135         for k, tpl in DELTA.items():
136             if key_lst[k]:
137                 sum_mv[0] += tpl[0]
138                 sum_mv[1] += tpl[1]
139
140         kk_img = kk_imgs[tuple(sum_mv)]
141
142         kk_rct.move_ip(sum_mv)
143         if check_bound(kk_rct) != (True, True):
144             kk_rct.move_ip(-sum_mv[0], -sum_mv[1])
145         vx, vy = calc_orientation(bb_rct, kk_rct, (vx, vy))
146         avx = vx*bb_accs[min(tmr//500, 9)]
147         avy = vy*bb_accs[min(tmr//500, 9)]
148         bb_img = bb_imgs[min(tmr//500, 9)]
149         bb_rct.move_ip(avx, avy)
150         bb_rct.width = bb_img.get_rect().width
151         bb_rct.height = bb_img.get_rect().height
152         yoko, tate = check_bound(bb_rct)
153         if not yoko:
154             vx *= -1
155         if not tate:
156             vy *= -1
157         screen.blit(kk_img, kk_rct)
```

```
158         screen.blit(bb_img, bb_rct)
159         pg.display.update()
160         tmr += 1
161         clock.tick(50)
162         if kk_rct.collidect(bb_rct):
163             print("game over")
164             gameover(screen)
165             return
166
167
168 if __name__ == "__main__":
169     pg.init()
170     main()
171     pg.quit()
172     sys.exit()
```